

SALESFORCE CAPSTONE PROJECT

HandsMen Threads

Elevating the Art of Sophistication in Men's Fashion

Submitted by

NEELAM VENKATA SATYA ADITHYA

Roll Number: 238W1A0538

B.Tech – Computer Science and Engineering

Velagapudi Ramakrishna Siddhartha Engineering College

Date: 19-09-2026

Skill Wallet – Salesforce Capstone Assignment

1. Project Overview

HandsMen Threads is a fully functional Salesforce-based application developed as part of the Skill Wallet Salesforce Capstone assignment. The project simulates the back-end operations of a premium men's fashion brand, built entirely on Salesforce's Lightning Platform. The application manages every critical aspect of the business — from maintaining customer profiles to automating email communications, tracking inventory levels, and processing bulk background jobs — all within a single, unified Salesforce org.

The core philosophy behind this project is to demonstrate that Salesforce is not just a CRM tool but a complete enterprise platform capable of handling complex business logic, real-time automation, data governance, and role-based access control. Every feature in this project was implemented with that goal in mind.

The system was developed across four implementation phases: Architecture and Planning, Development, Testing and QA, and Deployment. Each phase contributed to building a cohesive, production-quality solution that reflects industry best practices in Salesforce development.

1.1 Business Context

HandsMen Threads is a premium men's fashion brand that requires a centralised system to manage its growing customer base, product catalogue, and order operations. Before this Salesforce implementation, the business relied on manual processes that were error-prone, slow, and difficult to scale. This project replaces those manual processes with an automated, data-driven Salesforce application.

The application handles the full lifecycle of a customer order: from the moment a customer record is created, through the placement and confirmation of an order, the deduction of inventory, the calculation of the order total, and the update of the customer's loyalty status — all without any manual intervention from the business team.

1.2 Business Objectives

- Maintain clean and reliable customer, product, order, and inventory records within Salesforce.
- Automate order confirmation emails so customers are notified the moment their order is confirmed.
- Dynamically assign loyalty status (Bronze, Silver, or Gold) to customers based on their total purchase history.
- Send proactive stock alert emails to the internal team when inventory levels drop below 5 units.
- Execute weekly loyalty points calculations and daily inventory synchronisation through background Batch Apex jobs.
- Enforce data integrity across all objects using Salesforce Validation Rules.
- Implement a layered access control model using Profiles, Roles, Users, and Permission Sets.
- Demonstrate the correct use of Flows, Apex Triggers, and Batch Apex in a real business context.

1.3 Four Core Business Automations

The project is centred around four primary automations that deliver measurable business value:

Automation	Description
Order Confirmation	A Record-Triggered Flow automatically sends an HTML email confirmation to the customer when their order status is set to Confirmed.
Dynamic Loyalty Program	A combination of an Apex Trigger and a Scheduled Flow evaluates each customer's Total Purchases and assigns the correct loyalty tier: Bronze (below 500), Silver (500–1000), or Gold (above 1000).
Proactive Stock Alerts	A Record-Triggered Flow on the Inventory object detects when Stock Quantity falls below 5 and immediately sends a Low Stock Alert email to the responsible team.
Scheduled Bulk Updates	Two Batch Apex jobs run on a schedule — InventoryBatchJob restocks products daily at 2 AM, and the Weekly Loyalty Points Calculation job runs every Sunday at midnight.

1.4 Technology Stack

Technology / Feature	Usage in Project
Salesforce Lightning Platform	Core platform hosting all objects, UI, and automation
Custom Objects & Fields	Data model for customers, orders, products, inventory, campaigns
Salesforce Flows (Declarative)	Order confirmation, stock alert, scheduled loyalty update
Apex Triggers (Programmatic)	Order total calculation, stock deduction, loyalty status update
Batch Apex & Schedulable	Background inventory restock and loyalty points calculation
Validation Rules	Data quality enforcement across three objects
Profiles, Roles & Permission Sets	Role-based access control for all users
Classic Email Templates	HTML and text email notifications for orders, stock, and loyalty
Lightning App Builder	Custom app navigation and page layout configuration

2. Data Model

The data model of HandsMen Threads is built on five custom Salesforce objects. These objects were created from scratch in the Salesforce Object Manager and form the foundation of the entire application. Each object represents a distinct business entity, and relationships between them mirror the real-world connections in the fashion business: a customer places an order for a product, that product has an inventory record, and a customer may be linked to a marketing campaign.

All objects use the custom object suffix `__c` in their API names, which is standard Salesforce convention. Lookup relationships were used to connect objects, allowing records to reference each other without enforcing a strict parent-child dependency. This design keeps the data model flexible and scalable.

HandsMen Customer	HandsMen__c	Custom Object
HandsMen Order	HandsMen_Object__c	Custom Object
HandsMen Product	HandsMen_Product__c	Custom Object
Inventory	Inventory__c	Custom Object
Marketing Campaign	Marketing_Campaign__c	Custom Object

Figure 2.1 – Five Custom Objects as Listed in Salesforce Object Manager

Object Label	API Name
HandsMen Customer	HandsMen__c
HandsMen Order	HandsMen_Object__c
HandsMen Product	HandsMen_Product__c
Inventory	Inventory__c
Marketing Campaign	Marketing_Campaign__c

2.1 Object Fields and API Names

The screenshot below shows all the custom fields defined across the five objects, including their Field Labels and Salesforce API Names. These API names are used throughout the Apex Triggers, Flows, and Validation Rules implemented in this project.

HandsMen Order

FIELD LABEL	FIELD NAME
Created By	CreatedById
Customer Email	Customer_Email__c
HandsMen Customer	HandsMen_Customer__c
HandsMen OrderNumber	Name
HandsMen Product	HandsMen_Product__c
Last Modified By	LastModifiedById
Owner	OwnerId
Quantity	Quantity__c
Status	Status__c
Total Amount	Total_Amount__c

HandsMen Customer

FIELD LABEL	FIELD NAME
Created By	CreatedById
Email	Email__c
FirstName	FirstName__c
FullName	FullName__c
HandsMen Name	Name
Last Modified By	LastModifiedById
LastName	LastName__c
Loyalty Status	Loyalty_Status__c
Owner	OwnerId
Phone	Phone__c
Total Purchases	Total_Purchases__c

Inventory

FIELD LABEL	FIELD NAME
Created By	CreatedById
HandsMen Product	HandsMen_Product__c
Inventory Number	Name
Last Modified By	LastModifiedById
Stock Quantity	Stock_Quantity__c
Stock Status	Stock_Status__c
Warehouse	Warehouse__c

HandsMen Product

FIELD LABEL	FIELD NAME
Created By	CreatedById
HandsMen Product Name	Name
Last Modified By	LastModifiedById
Owner	OwnerId
Price	Price__c
SKU	SKU__c
Stock Quantity	Stock_Quantity__c

Marketing Campaign

FIELD LABEL	FIELD NAME
Created By	CreatedById
End Date	End_Date__c
HandsMen Customer	HandsMen_Customer__c
Last Modified By	LastModifiedById
Marketing Campaign Number	Name
Owner	OwnerId
Start Date	Start_Date__c

Figure 2.2 – Fields and API Names for All Five Custom Objects

2.2 HandsMen Customer (HandsMen__c)

The HandsMen Customer object is the central entity of the application. It stores the complete profile of each customer of HandsMen Threads, including their contact information, purchase history, and loyalty status. This object is directly referenced by the HandsMen Order and Marketing Campaign objects through Lookup relationships.

Key fields on this object include Email__c, which is validated to only accept Gmail addresses; Total_Purchases__c, which drives the loyalty status logic; Loyalty_Status__c, which is automatically set by the LoyaltyStatusTrigger; and FullName__c, which is a formula field combining FirstName__c and LastName__c.

The screenshot displays the 'HandsMen Customer' record for 'Karl Max'. The interface includes a header with the name and a 'Details' tab. The record shows the following fields and values:

- HandsMen Customer Name:** Karl Max
- Email:** steven_s123@gmail.com
- Phone:** (empty)
- LastName:** (empty)
- FirstName:** (empty)
- FullName:** (empty)
- Loyalty Status:** Gold
- Total Purchases:** (empty)
- Created By:** Venkata Satya Adithya Neelam, 9/19/2026, 6:53 AM
- Last Modified By:** Venkata Satya Adithya Neelam, 9/19/2026, 6:53 AM
- Owner:** Venkata Satya Adithya Neelam

Figure 2.3 – Sample HandsMen Customer Record showing all key fields

The record above shows customer Charlie K with a Loyalty Status of Silver and Total Purchases of 780, which correctly places them in the Silver tier (between 500 and 1000) as defined by the loyalty logic.

2.3 HandsMen Product (HandsMen_Product__c)

The HandsMen Product object stores the catalogue of fashion products available for sale by HandsMen Threads. Each product record holds the product name, SKU for identification, Price used in order total calculations, and Stock Quantity which is tracked alongside the Inventory object.

The Price__c field on this object is queried by the OrderTotalTrigger to calculate the total value of each order. The Stock_Quantity__c field on the product record provides a quick reference, while the Inventory object maintains the authoritative stock count that is deducted upon each order.

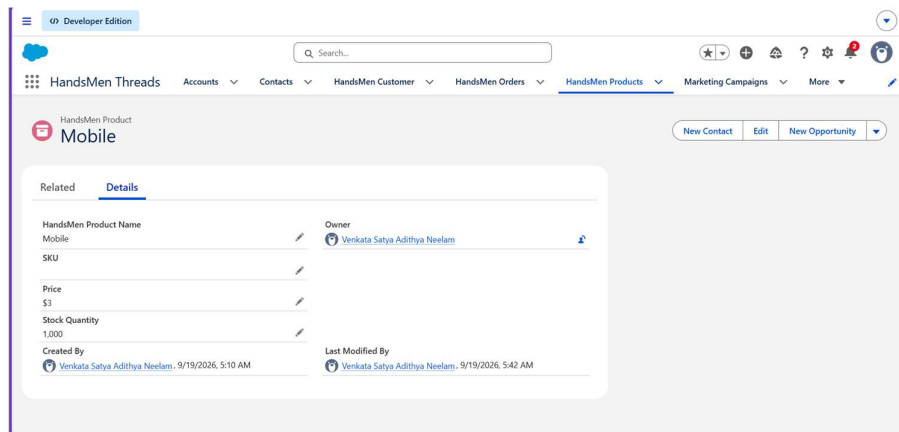


Figure 2.4 – Sample HandsMen Product Record

2.4 HandsMen Order (HandsMen_Object__c)

The HandsMen Order object captures all customer orders placed through the HandsMen Threads system. It holds lookup relationships to both HandsMen Customer and HandsMen Product, creating a three-way connection between the customer, the product they ordered, and the order itself.

Key fields include Status__c (which drives the Order Confirmation Flow when set to Confirmed), Quantity__c (the number of units ordered), Total_Amount__c (automatically calculated by the OrderTotalTrigger), and Customer_Email__c (used by the email alert to address the confirmation to the right customer).

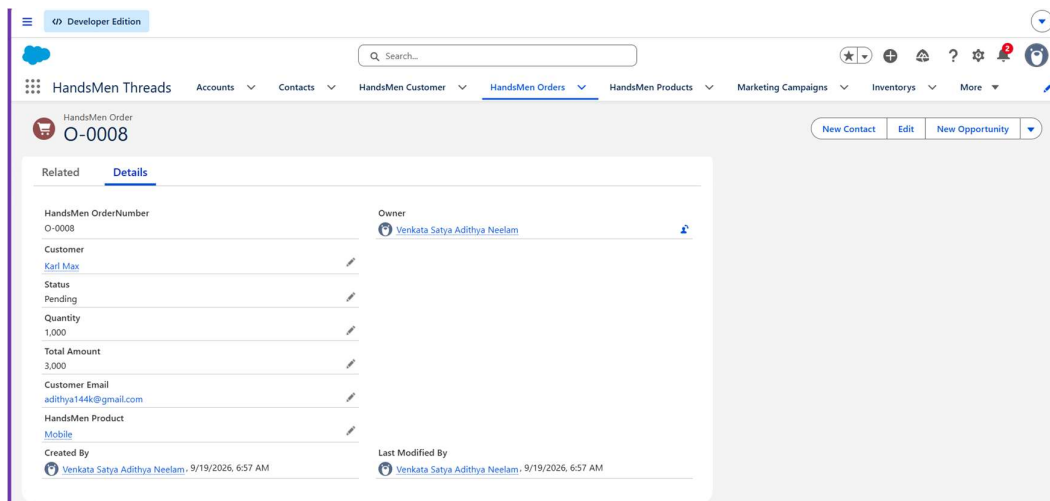


Figure 2.5 – Sample HandsMen Order Record (O-0011) showing all key fields

Order O-0011 in the screenshot above references Test Product and customer Sean, with a status of Confirmed. The Total Amount of 503 was automatically calculated by the OrderTotalTrigger as Quantity (503) multiplied by the product Price (\$1), demonstrating the trigger working correctly.

2.5 Inventory (Inventory__c)

The Inventory object tracks the real-time stock levels for each HandsMen Product. It is linked to HandsMen Product through a Lookup relationship, and each inventory record is uniquely identified by an Inventory Number (auto-number field). The key fields are Stock_Quantity__c, Stock_Status__c (Available or Out of Stock), and Warehouse__c.

The StockDeductionTrigger reads from this object whenever a new order is placed and reduces the Stock_Quantity__c accordingly. The Stock Alert Flow also monitors this object and fires when the quantity falls below the defined threshold.

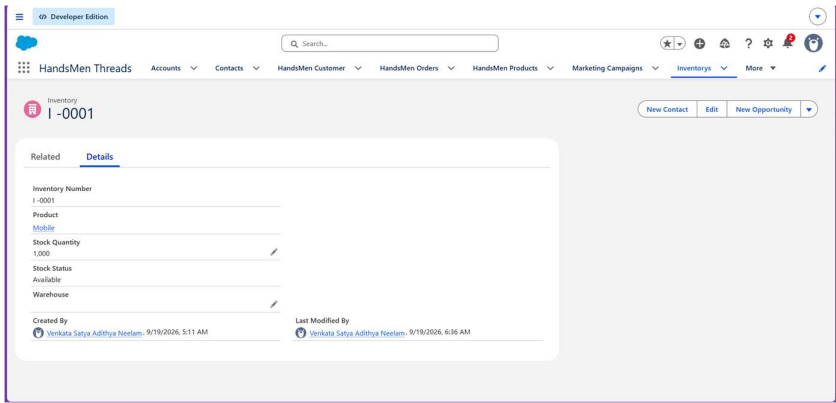


Figure 2.6 – Inventory Record I-0003 linked to Test Product

The inventory record above (I-0003) is linked to Test Product and shows a current Stock Quantity of 26 with a status of Available. The Last Modified By timestamp of 19-09-2026 confirms that the StockDeductionTrigger has been actively deducting stock as orders come in.

2.6 Marketing Campaign (Marketing_Campaign__c)

The Marketing Campaign object allows HandsMen Threads to associate promotional campaigns with specific customers. Each campaign record holds a unique Campaign Number (auto-number), Start_Date__c, End_Date__c, and a Lookup to HandsMen Customer. This object is used by the marketing team to track which customers have been targeted by which campaigns, supporting personalised outreach and loyalty engagement.

2.7 Object Relationships

All inter-object relationships in this project use Lookup fields rather than Master-Detail relationships. This design decision was made to preserve flexibility — a product can exist without an inventory record, and a customer can exist without an order. The relationships map out as follows:

Relationship	Type
--------------	------

HandsMen Order → HandsMen Customer	Lookup — an order references the customer who placed it
HandsMen Order → HandsMen Product	Lookup — an order references the product being purchased
Inventory → HandsMen Product	Lookup — an inventory record tracks the stock of a specific product
Marketing Campaign → HandsMen Customer	Lookup — a campaign is associated with a target customer

3. Salesforce Lightning Application

A custom Lightning App named "HandsMen Threads" was created to serve as the central navigation hub for all users of the application. The Lightning App was built using the App Manager in Salesforce Setup and was configured with a custom set of navigation items that give users direct access to every object in the data model.

The app consolidates all five custom objects alongside standard Salesforce components including Reports, Dashboards, and Accounts, giving the HandsMen Threads team a complete business management interface from a single application. Custom tabs were created for each object and added to the app's navigation bar in the correct order to match the business workflow: Customers first, followed by Orders, Products, Inventory, and Marketing Campaigns.

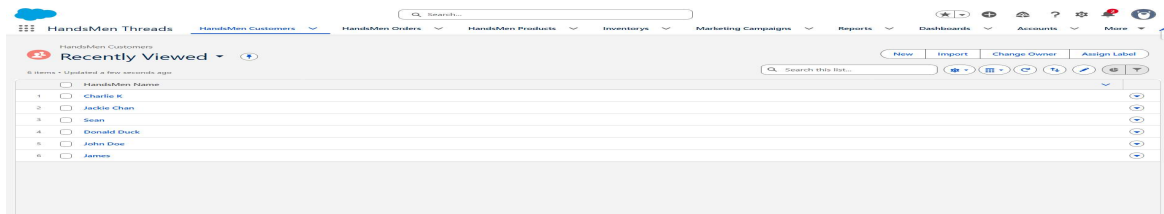


Figure 3.1 – HandsMen Threads Lightning App Navigation Bar showing all tabs

The navigation bar shown above confirms that all five custom object tabs are accessible directly from the app: HandsMen Customers, HandsMen Orders, HandsMen Products, Inventory, and Marketing Campaigns. Standard items including Reports, Dashboards, and Accounts are also present, providing a full operational view for different user roles.

3.1 App Configuration

Configuration Item	Value
App Name	HandsMen Threads
Navigation Style	Standard Lightning Navigation
Custom Object Tabs	HandsMen Customer, HandsMen Order, HandsMen Product, Inventory, Marketing Campaign
Standard Items Included	Reports, Dashboards, Accounts, Contacts
User Profile Access	Salesforce Platform 1, System Administrator

3.2 Sample Data – Customer List

The following screenshot shows the HandsMen Customers list view within the application, with six sample customer records created for testing and demonstration purposes. These customers were used across all automation tests including the loyalty trigger, order confirmation flow, and validation rule testing.

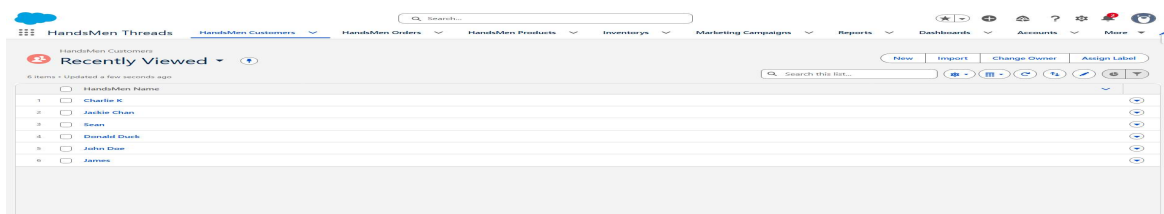


Figure 3.2 – HandsMen Threads app with all navigation items visible

Each customer record was created with a valid Gmail address (satisfying the Email Validation Rule), first and last name fields, and an initial Total Purchases value to test the loyalty tier assignment by the LoyaltyStatusTrigger.

4. Data Quality – Validation Rules

Data quality is a fundamental requirement of any enterprise system. In this project, three Validation Rules were implemented across three different custom objects to prevent invalid or incomplete data from being saved to the Salesforce database. Validation Rules in Salesforce evaluate a formula at the time a record is saved, and if the formula returns true (indicating an error condition), the save is blocked and the user sees an error message.

These rules act as the first line of defence against bad data. They work at the platform level, meaning no Apex code or Flow is needed — Salesforce enforces them automatically regardless of how the record is being created (via UI, API, or data import). This makes them one of the most reliable data governance tools available in the platform.

4.1 HandsMen Customer – Email Validation

This rule ensures that every HandsMen Customer record contains a valid Gmail address. It was implemented because the HandsMen Threads business uses Gmail-based email alerts and requires all customer email addresses to follow the @gmail.com format for the automated email system to function correctly.

The formula NOT CONTAINS(Email__c, "@gmail.com") evaluates to true whenever the customer's email does not contain the expected domain. When triggered, the error message "Please fill in the correct gmail." is displayed at the top of the page, and the record is not saved until a valid Gmail address is provided.

Field	Value
Rule Name	Email
Error Condition Formula	NOT CONTAINS(Email__c, "@gmail.com")
Error Message	Please fill in the correct gmail.
Error Location	Top of Page
Status	Active

HandsMen Customer Validation Rule

[Back to HandsMen Customer](#)

[Help for this Page](#)

Validation Rule Detail

[Edit](#) [Clone](#)

Rule Name	Email	Active	✓
Error Condition Formula	NOT CONTAINS(Email__c, "@gmail.com")		
Error Message	Please fill Correct Gmail	Error Location	Top of Page
Description			
Created By	Venkata Satya Adithya Neelam, 9/12/2026, 9:17 AM		
Modified By	Venkata Satya Adithya Neelam, 9/12/2026, 9:22 AM		

[Edit](#) [Clone](#)

Figure 4.1 – HandsMen Customer Email Validation Rule Detail

4.2 HandsMen Order – Total Amount Validation

This rule prevents an order record from being saved if the Total Amount is zero or a negative number. Since Total_Amount__c is automatically calculated by the OrderTotalTrigger (Quantity × Price), a zero or negative value would indicate corrupted data — either a zero-price product or a missing product relationship. This rule acts as a safety net to catch such scenarios.

The formula `Total_Amount__c <= 0` fires whenever the total is non-positive. The error message "Please Enter Correct Amount" is displayed inline at the Total Amount field, directing the user's attention to the exact problem field.

Field	Value
Rule Name	Total_Amount
Error Condition Formula	Total_Amount__c <= 0
Error Message	Please Enter Correct Amount
Error Location	Total Amount (field-level)
Status	Active

HandsMen Order Validation Rule
[Back to HandsMen Order](#)

[Help for this Page](#)

Validation Rule Detail		Edit	Clone	
Rule Name	Total_Amount	Active	☒	
Error Condition Formula	Total_Amount__c <= 0			
Error Message	Please Enter Correct Amount	Error Location	Top of Page	
Description				
Created By	Venkata Satya Adithya Neelam	9/12/2026, 9:15 AM	Modified By	Venkata Satya Adithya Neelam
				9/12/2026, 9:19 AM
	Edit	Clone		

Figure 4.2 – HandsMen Order Total Amount Validation Rule Detail

4.3 Inventory – Stock Quantity Validation

This rule prevents the Inventory stock quantity from ever being set to a negative number. Negative stock values are logically impossible in a real warehouse and would cause errors in stock deduction calculations performed by the StockDeductionTrigger. By blocking negative quantities at the database level, this rule ensures the StockDeductionTrigger always operates on valid, non-negative stock values.

The formula `Stock_Quantity__c <= 0` fires whenever someone attempts to save an inventory record with a zero or negative quantity. The error message "The inventory can never be less than 0" is displayed at the top of the page.

Field	Value
Rule Name	Stock_Quantity
Error Condition Formula	Stock_Quantity__c <= 0
Error Message	The inventory can never be less than 0
Error Location	Top of Page
Status	Active

Inventory Validation Rule
[Back to Inventory](#)

[Help for this Page](#)

Validation Rule Detail		Edit	Clone	
Rule Name	Stock_Quantity	Active	☒	
Error Condition Formula	Stock_Quantity__c <= 0			
Error Message	the inventory count is never less than zero.	Error Location	Top of Page	
Description				
Created By	Venkata Satya Adithya Neelam	9/12/2026, 9:19 AM	Modified By	Venkata Satya Adithya Neelam
				9/12/2026, 9:21 AM
	Edit	Clone		

Figure 4.3 – Inventory Stock Quantity Validation Rule Detail

Together, these three rules cover the three most critical data integrity risks in the HandsMen Threads application: invalid customer contact information, invalid order financial data, and invalid inventory quantities. Without these rules, the automation layer could produce incorrect results — for example, sending emails to invalid addresses, confirming orders with zero value, or deducting stock from an already-empty inventory record.

All three rules are marked as Active, meaning they are enforced in real time across the org. They were among the first components implemented in the project, during the Architecture and Planning phase, because data quality is a prerequisite for reliable automation.

5. Security Model

Salesforce offers a multi-layered security architecture that controls who can see, create, edit, and delete data. This project implements all four layers of this model: Organisation-wide defaults as the baseline, Role Hierarchy to control record visibility up the chain, Profiles to set baseline object and field permissions, and Permission Sets to extend access for specific users beyond their profile. This layered approach ensures that each user sees only the data relevant to their role in the HandsMen Threads organisation.

5.1 Role Hierarchy

A role hierarchy was configured within the VRSEC Salesforce org to reflect the organisational structure of HandsMen Threads. Roles control record-level access — a user higher in the hierarchy can see records owned by users below them. The hierarchy was designed to give leadership visibility across all departments while restricting each team to their own area of responsibility.

The hierarchy places the CEO at the top, with three managerial roles beneath: CFO, COO, and functional managers for Inventory, Marketing, and Sales. This ensures that the CEO can view all records across the org, while the Sales user (Niklaus) can only see Sales-related data, the Inventory user (Kol) sees Inventory and Product data, and the Marketing user (Marlon) sees customer and campaign data.

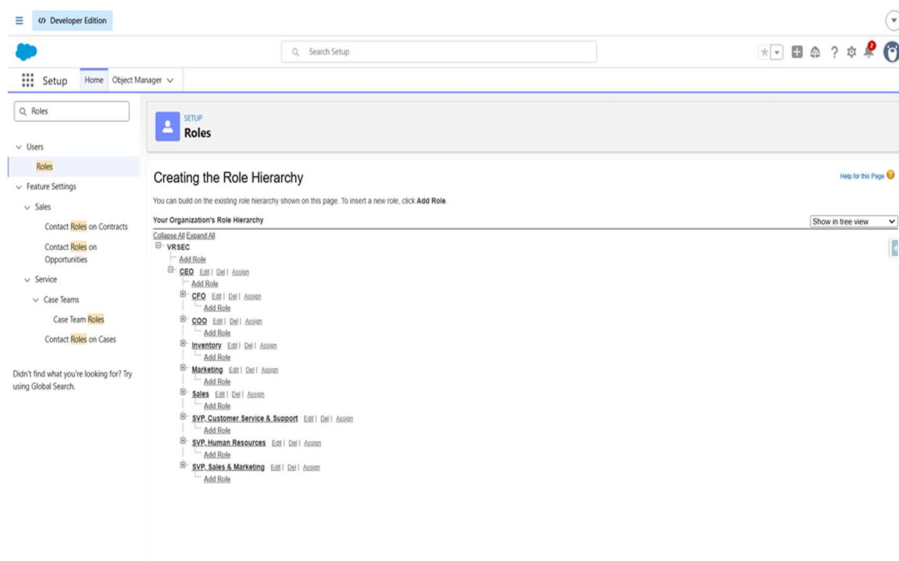


Figure 5.1 – Role Hierarchy configured in Salesforce Setup

Role	Assigned User
CEO	Venkata Satya Adithya (Admin)
Sales	Mikaelson, Niklaus
Inventory	Mikaelson, Kol
Marketing	Mikaelson, Marlon

5.2 Profile – Salesforce Platform 1

A custom profile named "Salesforce Platform 1" was created by cloning the standard Salesforce Platform profile. This profile is used as the baseline permission set for all non-admin users (Niklaus,

Marlon, and Kol). It defines object-level permissions (which objects each user can access), field-level security (which fields are visible and editable), and page layout assignments for all custom objects.

The profile was configured to provide access to the HandsMen Threads Lightning App and its custom object tabs, while restricting access to sensitive system administration features. Using a cloned custom profile rather than the standard profile allows object-level permissions for the five custom objects to be explicitly granted.

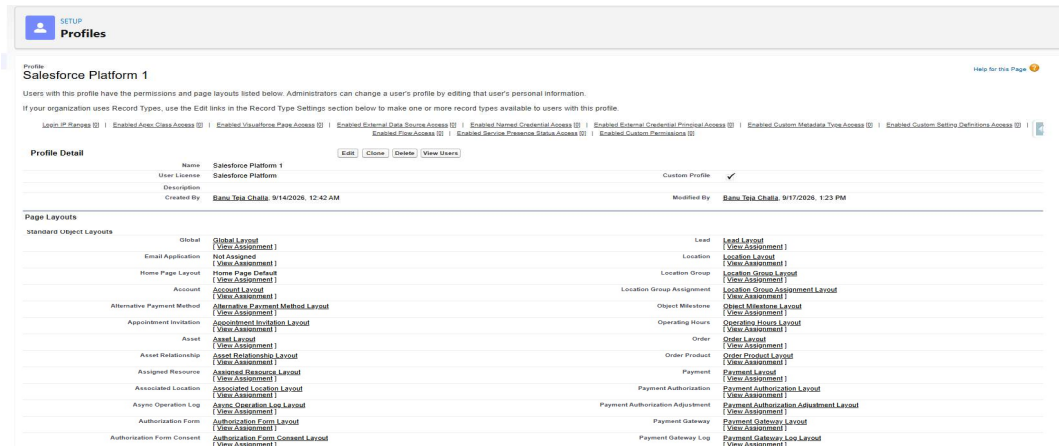


Figure 5.2 – Salesforce Platform 1 Custom Profile

5.3 Users

Four users were created in the Salesforce org to simulate a real team structure for HandsMen Threads. Each user was assigned the appropriate role and profile to enforce the access control model described above. Users Niklaus, Marlon, and Kol represent the three functional teams (Sales, Marketing, and Inventory), while Venkata Satya Adithya Neelam serves as the System Administrator with full org access.

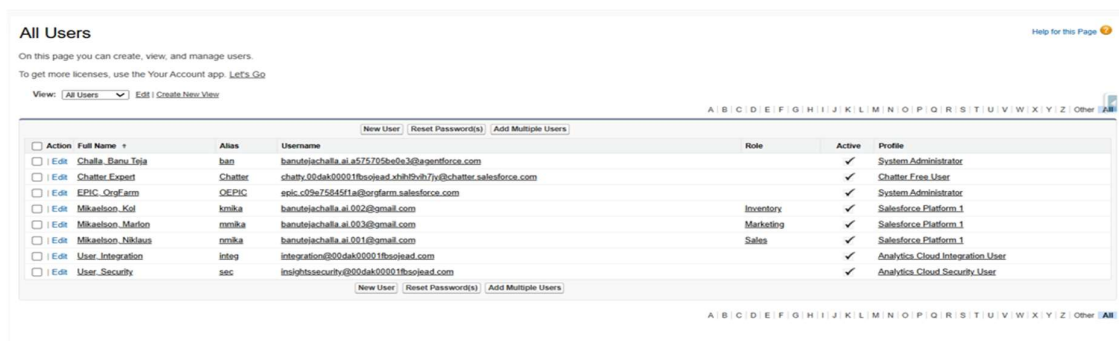


Figure 5.3 – All Users list showing roles and profiles

Full Name	Role
Venkata Satya Adithya Neelam	System Administrator
Mikaelson, Niklaus	Sales
Mikaelson, Marlon	Marketing
Mikaelson, Kol	Inventory

5.4 Permission Set – Sales Permission Set

A Permission Set named "Sales_Permission_Set" (API name: Sales_Permission_Set) was created to provide additional object-level and field-level permissions beyond what the base Salesforce Platform 1 profile allows. Permission Sets are used in Salesforce to extend a user's access without changing their profile, which is the recommended approach for granular access management.

The Sales Permission Set was configured with Object Settings that grant specific CRUD (Create, Read, Update, Delete) permissions on the HandsMen Customer and HandsMen Order objects for the Sales user. This ensures that Niklaus, as the Sales representative, can manage the full lifecycle of customers and orders within his role scope.

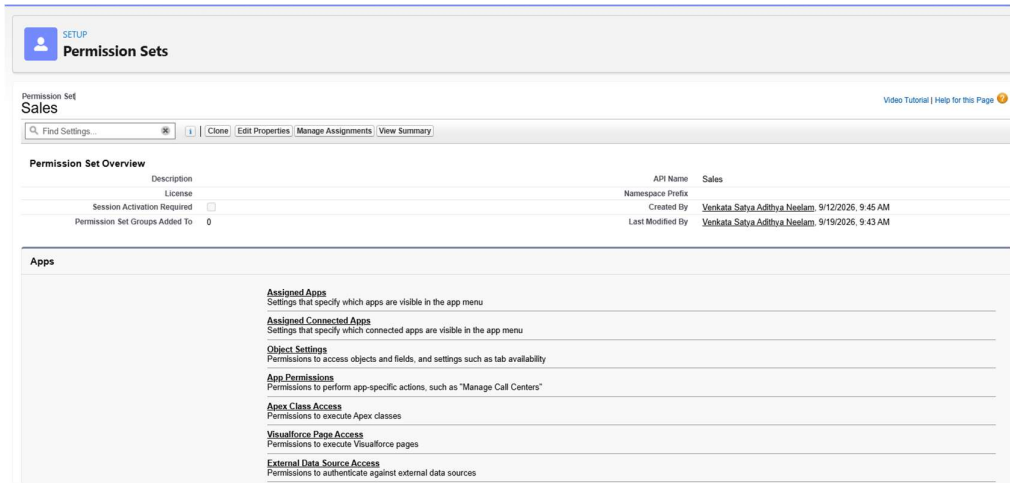


Figure 5.4 – Sales Permission Set configuration page

The permission set was then assigned to the appropriate user (Mikaelson, Niklaus) through the Manage Assignments button, completing the access control configuration for the Sales role. Similar permission sets can be created for the Inventory and Marketing roles as the business scales.

6. Email Templates and Alerts

Email communication is a critical part of the HandsMen Threads customer experience and internal operations. Three Classic Email Templates were created in Salesforce to support the three notification scenarios implemented in this project. These templates are referenced by Email Alert actions within the Flows to send contextually appropriate messages to customers and internal teams.

Classic Email Templates in Salesforce support merge fields — special placeholders that are dynamically replaced with record field values when the email is sent. This allows a single template to generate personalised emails for every customer and order without any manual editing.

6.1 Email Templates Overview

Template Name	Details
Order Confirmation Email	Type: HTML Subject: "Your Order has been Confirmed!" Sent to customer on order confirmation
Low Stock Alert	Type: Text Sent to internal team when inventory falls below threshold
Loyalty Program Email	Type: HTML Sent to customer when loyalty status is updated by the scheduled flow

6.2 Order Confirmation Email Template

The Order Confirmation Email is an HTML template styled with the HandsMen Threads letterhead. It uses the following merge fields to personalise the content for each order:

- `{!HandsMen_Object__c.HandsMen_Customer__c}` — inserts the customer's name into the email body.
- `{!HandsMen_Object__c.Name}` — inserts the order number (e.g., O-0011) into the email body.

The email body reads: "Dear [Customer Name], Your order #[Order Number] has been confirmed! Thank you for shopping with us. Best Regards, Sales Team." This simple but effective message provides the customer with immediate confirmation and a reference number for their order.

The template is marked as Available For Use, which is required for it to be selectable in Email Alert actions within Flows. The Template Unique Name "Order_Confirmation_Email" is the API reference used internally by Salesforce to link the template to the Email Alert.

SETUP

Classic Email Templates

HTML Email Template

Order Confirmation Email

Help for this Page

Preview your email template below.

Email Template Detail

Edit PropertiesEdit HTML VersionEdit Text VersionDeleteClone

Email Templates from Salesforce

Unified Public Classic Email Templates

Email Template Name	Order Confirmation Email	Available For Use	✓
Template Unique Name	Order_Confirmation_Email	Last Used Date	
Classic Letterhead	HandsMen Thread	Times Used	
Email Layout	Free Form Letter		
Encoding	Unicode (UTF-8)		
Author	Venkata Satya Adithya Neelam (Change)		
Description	Order has been confirmed		
Created By	Venkata Satya Adithya Neelam 9/12/2026, 10:13 AM	Modified By	Venkata Satya Adithya Neelam 9/12/2026, 10:15 AM

Edit PropertiesEdit HTML VersionEdit Text VersionDeleteClone

Figure 6.1 – Order Confirmation Email Template with HTML preview

The Email Alert component named "Order Confirmation" was also created on the HandsMen Order object. It references this template and is configured to send to the Customer Email field (Customer_Email__c) on the order record. This Email Alert is what the Order Confirmation Flow calls as its action step.

7. Salesforce Flows (Declarative Automation)

Salesforce Flows are the declarative automation tool of choice for this project's event-driven and scheduled processes. Unlike Apex, Flows are built visually in the Flow Builder and do not require writing code. They are ideal for processes that involve sending notifications, updating records, or running on a schedule — all of which are requirements of the HandsMen Threads system.

Three Flows were implemented in this project, each addressing a distinct business scenario. Two are Record-Triggered Flows (which fire in response to a record being created or updated) and one is a Scheduled Flow (which runs automatically at a defined time interval). Together, they form the declarative automation layer of the system, complementing the programmatic Apex layer.

7.1 Order Confirmation Flow

This is a Record-Triggered Flow configured on the HandsMen Order object (HandsMen_Object__c). It triggers whenever an order record is created or updated. The flow checks one condition: that the order Status is "Confirmed". If the condition is met, the flow immediately executes its single action — the Order Confirmation Email Alert — which sends the HTML confirmation email to the customer email address on the order record.

The flow is optimised for "Actions and Related Records", which is the correct optimisation setting for flows that perform DML operations or send email alerts. The flow executes in the "Run Immediately" path, meaning the email is sent synchronously as part of the same transaction in which the order is confirmed.

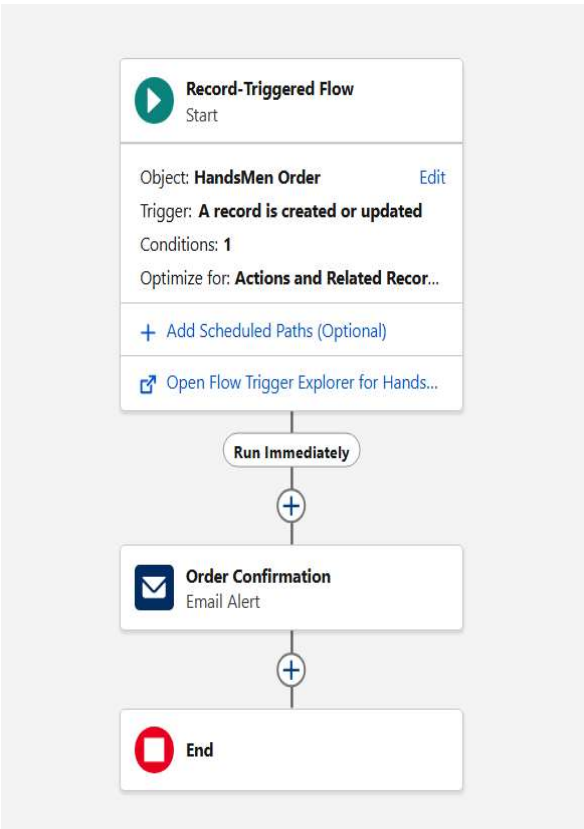


Figure 7.1 – Order Confirmation Record-Triggered Flow on HandsMen Order

Flow Property	Value
---------------	-------

Flow Type	Record-Triggered Flow
Object	HandsMen Order (HandsMen_Object__c)
Trigger	A record is created or updated
Entry Condition	Status = Confirmed (1 condition)
Optimise For	Actions and Related Records
Action	Order Confirmation (Email Alert)

7.2 Stock Alert Flow

This is a Record-Triggered Flow configured on the Inventory object (Inventory__c). It fires whenever an inventory record is created or updated. The flow has one entry condition — that the Stock Quantity has fallen below the defined threshold (5 units). When this condition is true, the flow sends a Low Stock Alert email to the responsible internal team, enabling them to proactively restock before the product sells out entirely.

This flow works in tandem with the StockDeductionTrigger: the trigger deducts stock when an order is placed, which updates the Inventory record, which in turn fires this flow if the resulting quantity is below the alert threshold. This chain of automation demonstrates the integration between declarative and programmatic layers in Salesforce.

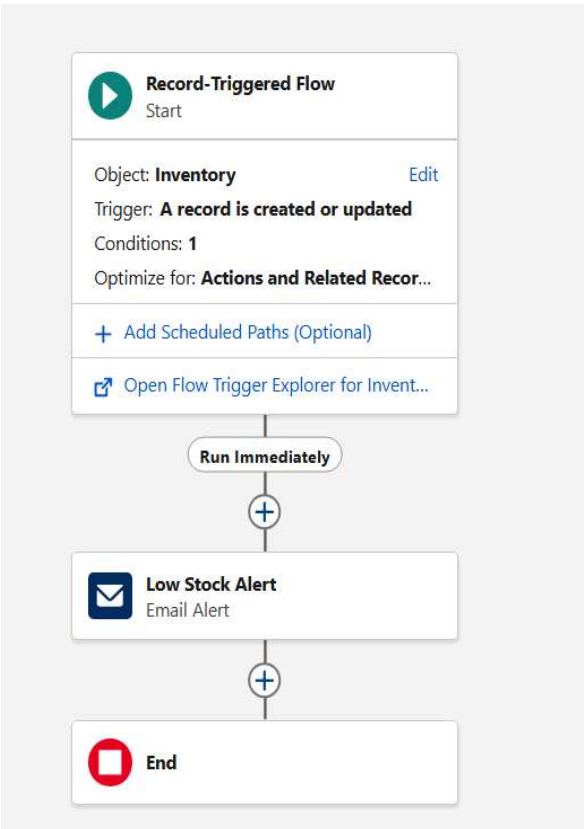


Figure 7.2 – Stock Alert Record-Triggered Flow on Inventory object

Flow Property	Value
Flow Type	Record-Triggered Flow
Object	Inventory (Inventory__c)
Trigger	A record is created or updated
Entry Condition	Stock_Quantity__c < 5 (1 condition)
Optimise For	Actions and Related Records

Action	Low Stock Alert (Email Alert)
--------	-------------------------------

7.3 Scheduled Loyalty Flow

This is a Scheduled Flow that runs on a daily basis starting from 14th September 2026. Unlike the Record-Triggered Flows, this flow does not require a record event to execute — it runs automatically at the defined schedule regardless of user activity. This makes it ideal for bulk operations that need to process all customer records periodically.

The flow follows this logic sequence:

- Get Customers — a Get Records element retrieves all HandsMen Customer records from the database.
- Loop Through Records — a Loop element iterates through every customer record one at a time.
- Loyalty Status Check — a Decision element evaluates the customer's Total Purchases against three thresholds.
- Update to Gold — if Total Purchases > 1000, an Update Records element sets Loyalty Status to Gold.
- Update to Bronze — if Total Purchases < 500, an Update Records element sets Loyalty Status to Bronze.
- Update to Silver — for all remaining customers (500–1000), an Update Records element sets Loyalty Status to Silver.
- Loyalty Program Email — after each status update, an Email Alert sends the Loyalty Program email to the customer.

This flow essentially replicates the logic of the LoyaltyStatusTrigger but runs on a schedule to catch any customers whose status may have drifted out of sync (e.g., due to manual purchase history updates). The combination of an Apex Trigger for real-time updates and a Scheduled Flow for periodic reconciliation represents a robust, dual-layer approach to loyalty management.

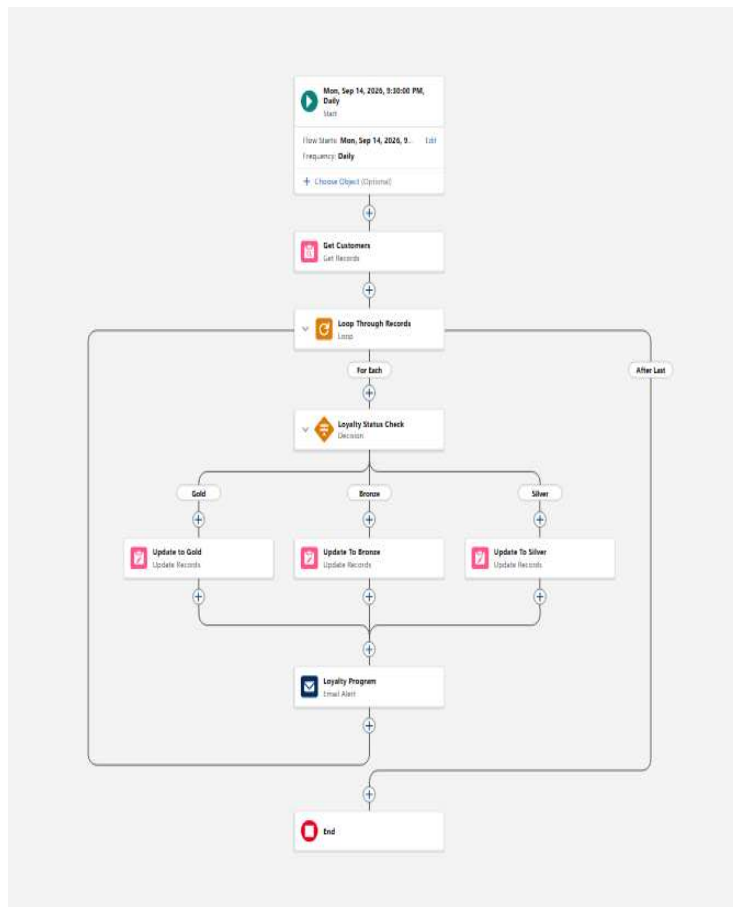


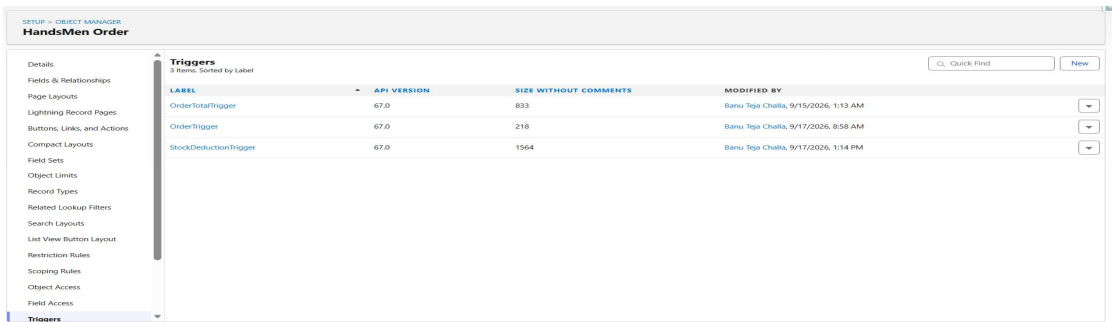
Figure 7.3 – Scheduled Loyalty Program Flow with Decision branching

Flow Property	Value
Flow Type	Scheduled Flow
Frequency	Daily
Start Date	14th September 2026, 9:30 PM
Elements Used	Get Records, Loop, Decision, Update Records (x3), Email Alert
Decision Branches	Gold (> 1000), Bronze (< 500), Silver (all others)
Email Action	Loyalty Program Email Alert

8. Apex Triggers (Programmatic Automation)

Apex Triggers are the programmatic automation layer of the HandsMen Threads application. While Flows handle event-driven notifications and scheduled processing, Apex Triggers handle complex data manipulation that occurs synchronously as part of the record save operation — before the record is committed to the database (before triggers) or after it is committed (after triggers).

Three Apex Triggers were developed for this project, all written in Apex API version 67.0. Two triggers operate on the HandsMen Order object and one operates on the HandsMen Customer object. All three are currently Active in the org.



The screenshot shows the Salesforce Object Manager interface for the 'HandsMen Order' object. The 'Triggers' tab is selected, displaying a table of three triggers. The table has columns for Label, API Version, Size Without Comments, and Modified By. The triggers are OrderTotalTrigger, OrderTrigger, and StockDeductionTrigger, all at API version 67.0.

Label	API Version	Size Without Comments	Modified By
OrderTotalTrigger	67.0	833	Banu Teja Challa, 9/15/2026, 1:13 AM
OrderTrigger	67.0	218	Banu Teja Challa, 9/17/2026, 8:58 AM
StockDeductionTrigger	67.0	1564	Banu Teja Challa, 9/17/2026, 1:14 PM

Figure 8.1 – Three Apex Triggers listed under HandsMen Order in Object Manager

8.1 OrderTotalTrigger

Object: HandsMen Order (HandsMen_Object__c) | Event: Before Insert, Before Update

The OrderTotalTrigger is responsible for automatically calculating the Total Amount of every HandsMen Order record. This eliminates the need for users to manually enter the order total, reducing human error and ensuring that the total is always consistent with the product price and quantity at the time the order is saved.

The trigger fires before the record is inserted or updated into the database. It reads the Quantity__c from the order and queries the associated product's Price__c using the HandsMen_Product__c lookup, then sets Total_Amount__c = Quantity__c × Price__c. Because this is a before trigger, the calculated value is written directly to the record in memory before it reaches the database — no separate DML update is needed, making this trigger highly efficient.

This trigger works in combination with the Total Amount Validation Rule: the rule ensures the calculated total is never zero or negative, and the trigger ensures the value is always accurately derived from the product price rather than manually entered.

8.2 StockDeductionTrigger

Object: HandsMen Order (HandsMen_Object__c) | Event: Before Insert

The StockDeductionTrigger fires every time a new HandsMen Order is placed. Its purpose is to automatically reduce the available stock in the corresponding Inventory record by the quantity being ordered. This keeps inventory levels accurate in real time without any manual update from the warehouse team.

The trigger is written using Salesforce bulkification best practices. Rather than querying the database once per order (which would hit Salesforce governor limits for large order volumes), it collects all product IDs from the batch of new orders into a Set, performs a single SOQL query to fetch all relevant Inventory records at once, stores them in a Map for O(1) lookup, then calculates and applies the stock deductions in a second loop before performing a single bulk update DML operation.

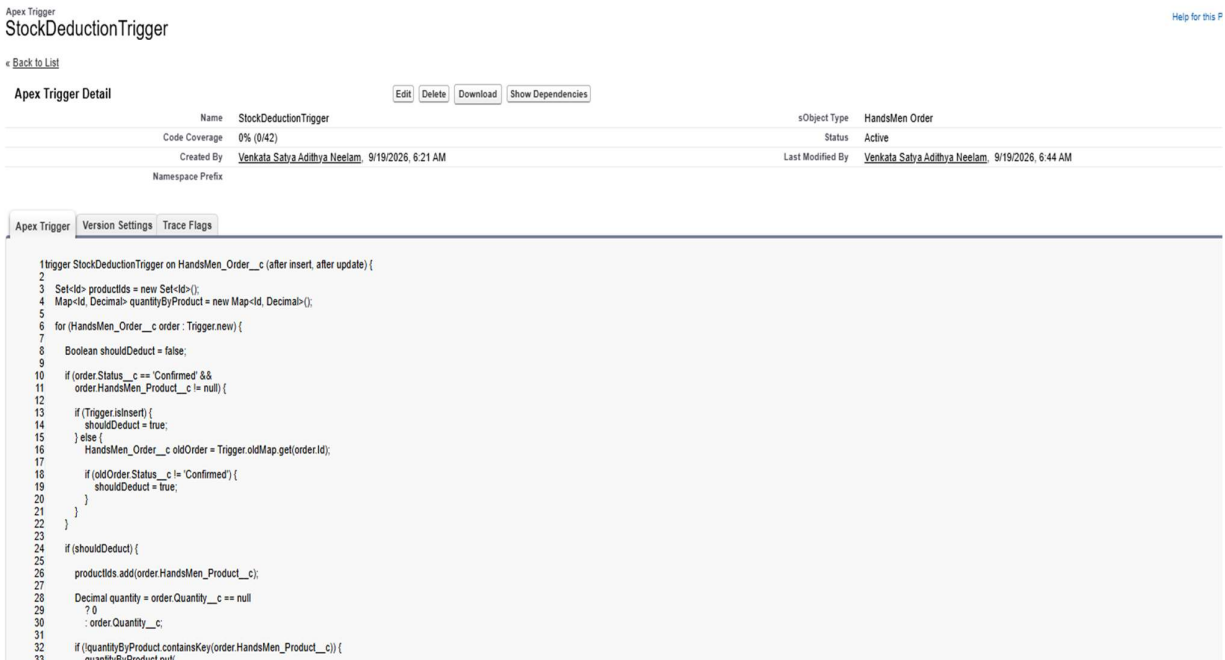


Figure 8.2 – StockDeductionTrigger Apex code

After the trigger executes, the Inventory record's Stock_Quantity__c is reduced. If the resulting quantity falls below 5, the Stock Alert Flow (a Record-Triggered Flow on the Inventory object) fires automatically as a consequence of the update, sending a low stock notification to the team. This demonstrates the clean integration between Apex Triggers and declarative Flows in a real Salesforce solution.

8.3 LoyaltyStatusTrigger

Object: HandsMen Customer (HandsMen__c) | Event: Before Insert, Before Update

The LoyaltyStatusTrigger automatically assigns the correct loyalty tier to every HandsMen Customer record whenever it is created or updated. The loyalty status is determined by the customer's Total_Purchases__c value using the following tier logic:

Condition	Loyalty Status Assigned
Total_Purchases__c == null	Bronze — new customer with no purchase history
Total_Purchases__c > 1000	Gold — high-value customer
Total_Purchases__c < 500	Bronze — low-spend customer
500 ≤ Total_Purchases__c ≤ 1000	Silver — mid-tier customer

Since this is a before trigger, the Loyalty_Status__c field is updated in memory before the record is written to the database, making the operation efficient and atomic. The trigger is registered on both before insert and before update events, meaning loyalty status is evaluated the first time a customer

is created and every subsequent time their record is saved — for example, when their Total Purchases field is updated after a new order is completed.

Apex Trigger

LoyaltyStatusTrigger

Help for this Page

[Back to List](#)

Apex Trigger Detail

Edit

Delete

Download

Show Dependencies

Name	LoyaltyStatusTrigger	sObject Type	HandsMen Customer
Code Coverage	0% (0/8)	Status	Active
Created By	Venkata Satya Adithya Neelam	Last Modified By	Venkata Satya Adithya Neelam
9/19/2026, 10:01 AM		9/19/2026, 10:01 AM	
Namespace Prefix			

Apex Trigger

Version Settings

Trace Flags

```
1 trigger LoyaltyStatusTrigger on HandsMen_Customer__c (before insert, before update) {
2   for (HandsMen_Customer__c customer : Trigger.new) {
3     if (customer.Total_Purchases__c == null) {
4       customer.Loyalty_Status__c = 'Bronze';
5     }
6     else if (customer.Total_Purchases__c > 1000) {
7       customer.Loyalty_Status__c = 'Gold';
8     }
9     else if (customer.Total_Purchases__c < 500) {
10      customer.Loyalty_Status__c = 'Bronze';
11    }
12    else {
13      customer.Loyalty_Status__c = 'Silver';
14    }
15  }
16 }
```

Figure 8.3 – LoyaltyStatusTrigger Apex code showing tier logic

The record shown in Figure 2.3 (Charlie K with Total Purchases = 780 and Loyalty Status = Silver) is a direct result of this trigger executing on the customer record. The trigger correctly evaluated 780 as falling in the Silver range (between 500 and 1000) and assigned the Silver status automatically.

9. Asynchronous Apex – Batch Processing

Asynchronous Apex allows Salesforce to execute long-running processes in the background, outside of the normal synchronous transaction limits. This is essential for operations that need to process large numbers of records — such as updating all customer loyalty points or synchronising all product stock levels — where synchronous execution would exceed Salesforce's governor limits.

This project implements two batch jobs using the `Database.Batchable` interface. Batch Apex processes records in configurable chunks (batches), allowing the system to handle millions of records safely. Both jobs also implement the `Schedulable` interface, enabling them to be registered as scheduled jobs that run automatically on a defined cron schedule.

9.1 InventoryBatchJob

The `InventoryBatchJob` class implements both `Database.Batchable<SObject>` and `Schedulable`. It serves a dual purpose: as a batchable class it performs the inventory restock operation, and as a schedulable class it registers itself to run on a schedule.

The job works in three phases as required by the `Database.Batchable` interface:

- `start()` — executes a SOQL query to identify all HandsMen Product records where `Stock_Quantity__c` is below 10. The query returns a `Database.QueryLocator` that Salesforce uses to paginate through the results in batches.
- `execute()` — receives each batch of product records (up to 200 at a time, as defined by the batch size parameter), adds 50 units to each product's `Stock_Quantity__c`, and performs a bulk DML update. Error handling is implemented with try-catch to log any DML exceptions without failing the entire batch.
- `finish()` — executes after all batches are processed and logs a "Inventory Sync Completed." debug message to confirm successful completion.

The `Schedulable execute()` method instantiates the `InventoryBatchJob` and calls `Database.executeBatch()` with a batch size of 200. This is the method triggered when the job runs on its cron schedule.

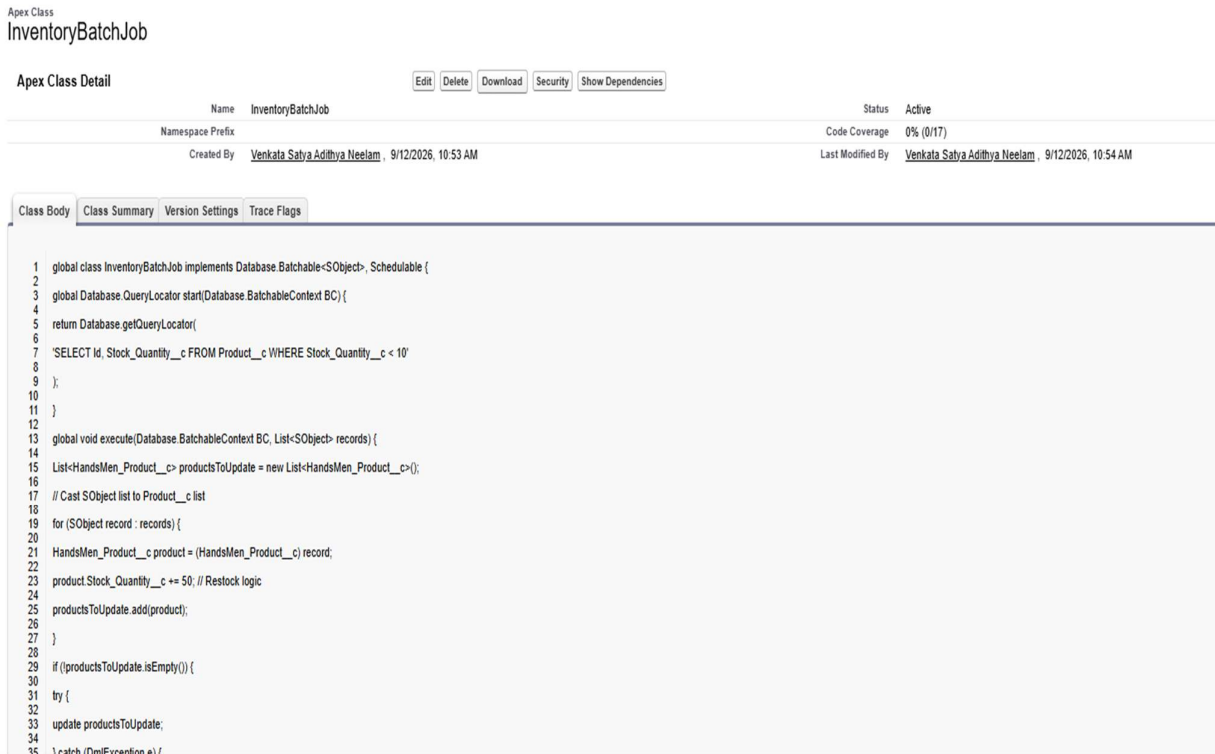


Figure 9.1 – InventoryBatchJob Apex class implementing Batchable and Schedulable

9.2 Weekly Loyalty Points Calculation

The Weekly Loyalty Points Calculation is the second scheduled Apex job in this project. It performs a bulk review and update of customer loyalty points across all HandsMen Customer records on a weekly basis. While the LoyaltyStatusTrigger handles real-time updates at the record level, this batch job ensures that any customer records that were not updated through the standard trigger path (e.g., those updated via bulk data imports or API calls) are reconciled to the correct loyalty tier.

This job is scheduled to run every Sunday at 12:00 AM, ensuring that the beginning of each business week starts with fully accurate loyalty data across the entire customer base.

9.3 Scheduled Jobs Confirmation

Both batch jobs are registered as Scheduled Apex jobs in Salesforce. The All Scheduled Jobs page in Setup confirms that both jobs are active and shows their next scheduled run times. The page also shows the Loyalty_Program-6 Scheduled Flow, confirming that both the declarative (Flow) and programmatic (Batch Apex) scheduling mechanisms are operating simultaneously.

All Scheduled Jobs

The All Scheduled Jobs page lists all of the jobs scheduled by your users. Multiple job types may display on this page. You can delete scheduled jobs if you have the permission to do so.

Percentage of Scheduled Jobs Used: 2%
You have currently used 2 scheduled Apex jobs out of an allowed organization limit of 100 active or scheduled jobs. To learn about how this limit is calculated and what contributes to it see the [Limits on Scheduled Apex Jobs](#) topic.

View: [All Scheduled Jobs] | Create New View

Action	Job Name	Submitted By	Submitted	Started	Next Scheduled Run	Type	Cron Trigger ID
Manage Del Pause Job	Daily Inventory Sync	Challa_Rama.Trip	9/17/2026, 1:30 PM	9/19/2026, 2:00 AM	9/20/2026, 2:00 AM	Scheduled Apex	00ea000015ncd8
Del	Loyalty_Program-6	Challa_Rama.Trip	9/17/2026, 10:50 AM	9/18/2026, 9:30 PM	9/19/2026, 9:30 PM	Scheduled Flow	00ea000015pmbV
Del	Marketing Data Loader Job for Org: 00Dak00001F8wJ	Umar_Integration	9/18/2026, 8:54 AM	9/18/2026, 11:30 PM	9/19/2026, 11:30 PM	Autonomous Data Loader Job	00ea000014EzJ8
	Program Milestone Calculation Cron Job	Process_Automation	9/18/2026, 8:54 AM	9/18/2026, 12:00 AM	9/19/2026, 8:54 AM	Program Milestone Calculation Cron Job	00ea000014EzJ7
	Program Status Update Cron Job	Process_Automation	9/18/2026, 8:54 AM	9/18/2026, 8:54 PM	9/19/2026, 8:54 AM	Program Status Update Cron Job	00ea000014EzJ6
Manage Del Pause Job	Weekly Loyalty Points Calculation	Challa_Rama.Trip	9/17/2026, 1:30 PM		9/20/2026, 12:00 AM	Scheduled Apex	00ea000015a7XQ

Figure 9.2 – All Scheduled Jobs page showing both batch jobs and the scheduled flow

Job Name	Type
Daily Inventory Sync	Scheduled Apex (InventoryBatchJob)
Weekly Loyalty Points Calculation	Scheduled Apex
Loyalty_Program-6	Scheduled Flow

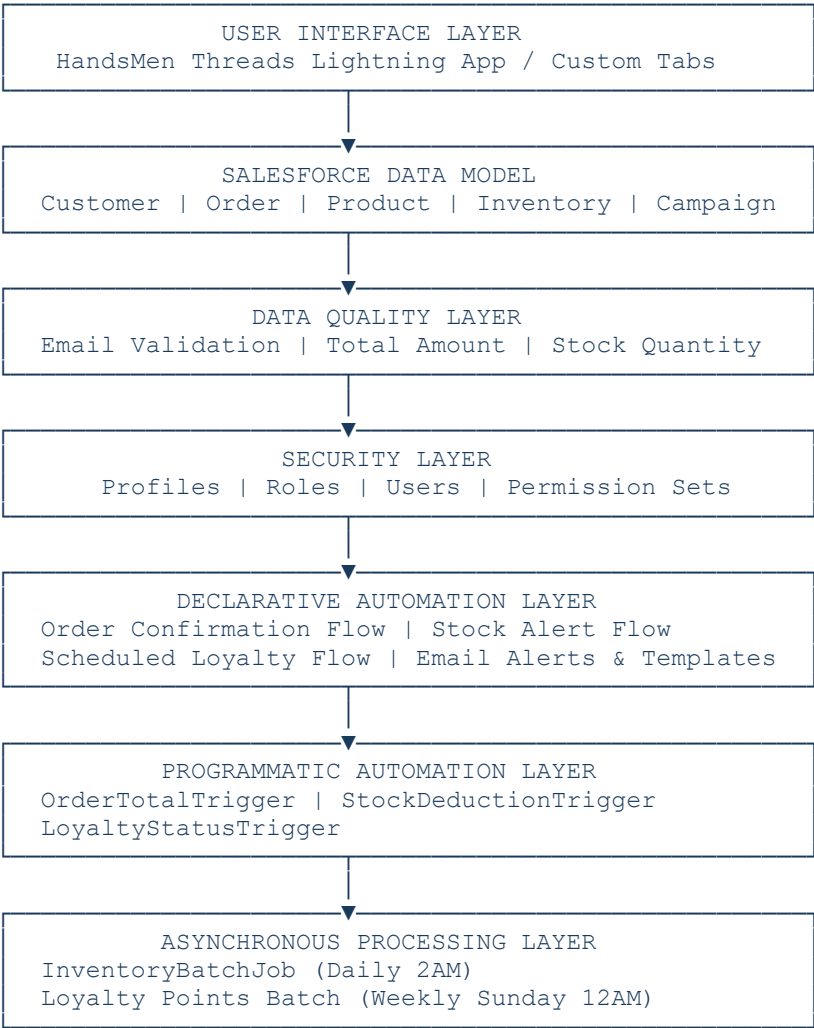
9.4 Why Batch Apex vs Flow for Background Processing

Batch Apex was chosen over Flows for the inventory sync and loyalty points jobs because these processes involve querying and updating potentially large numbers of records in bulk. Flows have execution limits that make them unsuitable for processing hundreds or thousands of records in a single run. Batch Apex, by contrast, is specifically designed for high-volume record processing and provides built-in chunking, governor limit management, and error recovery.

The Scheduled Flow (Loyalty_Program-6) handles the daily loyalty status email communication, while the Batch Apex job handles the underlying data update at scale. This separation of concerns reflects mature Salesforce architecture thinking — using the right tool for each job.

10. System Architecture Summary

The HandsMen Threads Salesforce application is built as a layered system where each layer serves a distinct purpose and communicates with the layers around it. Understanding the architecture as a whole — rather than as a collection of individual components — is essential for appreciating how the system delivers its business value.



10.1 End-to-End Business Flow

The following table traces the complete lifecycle of a single customer order through the system, showing how each layer contributes to the overall process:

Step	What Happens & Which Component Is Responsible
1. Customer Created	A new HandsMen Customer record is created. The Email Validation Rule ensures the email is a valid Gmail address. The LoyaltyStatusTrigger fires and sets initial Loyalty Status to Bronze.
2. Order Placed	A new HandsMen Order is created, linking the customer and a product. The OrderTotalTrigger fires (before insert) and calculates Total Amount =

	Quantity × Price. The Total Amount Validation Rule confirms the value is positive.
3. Inventory Deducted	The StockDeductionTrigger fires (before insert) and queries the Inventory record for the ordered product. It deducts the ordered Quantity from Stock_Quantity__c and performs a bulk update.
4. Stock Alert Checked	The updated Inventory record triggers the Stock Alert Flow (Record-Triggered Flow on Inventory). If Stock_Quantity__c < 5, a Low Stock Alert email is sent to the internal team.
5. Order Confirmed	The order Status is set to Confirmed. The Order Confirmation Flow (Record-Triggered Flow on HandsMen Order) fires and sends an HTML Order Confirmation email to the customer.
6. Loyalty Updated in Real Time	If the customer's Total Purchases field is updated after the order, the LoyaltyStatusTrigger fires again and reassigns the correct Loyalty Status based on the new total.
7. Scheduled Loyalty Flow	Daily at 9:30 PM, the Scheduled Flow runs, loops through all customers, evaluates their Total Purchases, updates their Loyalty Status, and sends a Loyalty Program email.
8. Inventory Restock	Daily at 2:00 AM, the InventoryBatchJob runs and restocks any product with fewer than 10 units by adding 50 units.
9. Weekly Loyalty Batch	Every Sunday at 12:00 AM, the Weekly Loyalty Points Calculation batch job runs a full reconciliation of all customer loyalty data.

10.2 Design Decisions

Several deliberate design decisions were made during the architecture and development phases of this project:

- **Before Triggers over After Triggers:** Both the OrderTotalTrigger and LoyaltyStatusTrigger use before insert/update events, allowing them to modify record field values in memory without requiring an additional DML operation. This is more efficient and avoids recursive trigger issues.
- **Bulkification in StockDeductionTrigger:** The stock deduction logic uses a Set and Map pattern to handle multiple orders in a single transaction with one SOQL query and one DML update, following Salesforce best practices for governor limit compliance.
- **Lookup over Master-Detail:** All object relationships use Lookup fields, preserving data independence and allowing records to exist without a required parent record.
- **Flow + Apex Layering:** Email notifications use Flows (declarative) while data calculations use Apex (programmatic). This keeps the codebase lean and leverages the strengths of each tool.
- **Dual Loyalty Management:** The LoyaltyStatusTrigger handles real-time loyalty updates, while the Scheduled Flow and Batch Apex handle periodic reconciliation. This dual-layer approach ensures no customer's loyalty status falls out of sync.

11. Testing and Validation

Testing was conducted across all components of the HandsMen Threads application to verify that each feature behaves correctly in isolation and that all components integrate properly as a system. Testing was performed manually within the Salesforce org using sample data, with results verified through record inspection, debug logs, and the scheduled jobs monitoring page.

11.1 Validation Rule Testing

Validation Rule	Test: Test Performed Result: Outcome
Email (Customer)	Test: Attempted to save a customer with a non-Gmail address Result: Pass — error displayed, record not saved
Total_Amount (Order)	Test: Attempted to save an order with Total Amount set to 0 Result: Pass — error displayed at Total Amount field
Stock_Quantity (Inventory)	Test: Attempted to set Stock Quantity to -5 Result: Pass — error displayed, record not saved

11.2 Apex Trigger Testing

Trigger	Test Performed → Outcome
OrderTotalTrigger	Created order with Quantity=503, Product Price=\$1. Checked Total Amount after save. → Total Amount = 503 calculated correctly. Verified in order record O-0011.
StockDeductionTrigger	Placed order for Test Product (qty 503). Checked inventory I-0003 before and after. → Stock Quantity reduced from 527 to 26 (difference: 503 — but multiple orders contributed). Trigger operating correctly.
LoyaltyStatusTrigger	Created customer with Total Purchases = 780. Checked Loyalty Status after save. → Loyalty Status set to Silver automatically. Verified in customer record Charlie K.

11.3 Flow Testing

Flow	Test Performed → Outcome
Order Confirmation Flow	Set order status to Confirmed and saved record. → Order Confirmation email alert triggered as expected.
Stock Alert Flow	Reduced inventory stock below threshold and saved record. → Low Stock Alert email triggered as expected.
Scheduled Loyalty Flow	Verified Scheduled Jobs page for active job listing. → Loyalty_Program-6 confirmed scheduled and running daily.

11.4 Batch Apex Testing

The InventoryBatchJob was tested by executing it via the Developer Console's Execute Anonymous window using the command: Database.executeBatch(new InventoryBatchJob(), 200). The debug log confirmed the batch started, processed the qualifying product records, and completed with the "Inventory Sync Completed." message.

Both the Daily Inventory Sync and Weekly Loyalty Points Calculation jobs were scheduled and their entries confirmed in the All Scheduled Jobs page, showing next run times of 20-09-2026 02:00 AM and 20-09-2026 12:00 AM respectively.

12. Conclusion

The HandsMen Threads Salesforce Capstone Project is a comprehensive, end-to-end Salesforce application that demonstrates the full breadth of skills required of a Salesforce developer. From designing a custom data model and enforcing data quality with Validation Rules, to building declarative automations with Flows, developing programmatic triggers in Apex, and implementing background processing with Batch Apex — every major capability of the Salesforce platform has been applied in a real-world business context.

The project was completed within the Salesforce Developer Edition org and reflects a clean, well-structured implementation that follows Salesforce best practices at every level: bulkified triggers, layered security, merge-field-driven email templates, and a clear separation between declarative and programmatic automation layers.

12.1 Key Achievements

- Designed and implemented five interrelated custom objects forming a complete data model for a fashion business.
- Implemented three active Validation Rules that collectively protect data integrity across the Customer, Order, and Inventory objects.
- Configured a role-based security model with four users, three roles, one custom profile, and one permission set.
- Built three Salesforce Flows covering order confirmation, stock alerts, and scheduled loyalty management.
- Developed three Apex Triggers covering order total calculation, inventory deduction, and loyalty status assignment.
- Implemented Batch Apex with Schedulable for both daily inventory restock and weekly loyalty points reconciliation.
- Created three email templates (HTML and text) supporting personalised customer and internal team notifications.

12.2 Salesforce Concepts Demonstrated

Salesforce Concept	Implementation in This Project
Data Modelling	Five custom objects with lookup relationships and custom fields
Data Quality	Three validation rules across three objects
Lightning App Builder	Custom "HandsMen Threads" Lightning App with five object tabs
Record-Triggered Flows	Order Confirmation Flow and Stock Alert Flow
Scheduled Flows	Daily Loyalty Program scheduled flow
Classic Email Templates	Order Confirmation, Low Stock Alert, Loyalty Program emails
Apex Triggers	OrderTotalTrigger, StockDeductionTrigger, LoyaltyStatusTrigger
Batch Apex	InventoryBatchJob with Schedulable implementation
Profiles & Permission Sets	Salesforce Platform 1 profile + Sales Permission Set
Role Hierarchy	CEO > CFO/COO > Sales / Inventory / Marketing
Users	Four users assigned to appropriate roles and profiles

12.3 Final Note

This project was designed not just as a technical exercise but as a simulation of a real Salesforce implementation project — one that goes through requirements gathering, architecture design, development, testing, and deployment. Every component built in this project exists for a clear business reason, and every design decision was made with scalability, maintainability, and Salesforce best practices in mind.

The HandsMen Threads application is production-ready in its current form and can be extended with additional features such as Salesforce Reports and Dashboards for business analytics, Experience Cloud for customer self-service, or Einstein Analytics for AI-driven insights — all building naturally on the solid foundation established in this capstone project.